

REFERENCE TABLES FOR ALL QUESTIONS

Use the following table structures while answering the questions. Assume the columns contain appropriate data types.

Table: employees

employee_id	employee_name	department_id	salary	joining_date	city
PK		FK → departments.department_id			

Table: departments

department_id	department_name
PK	

Table: projects

project_id	project_name	employee_id	project_cost	start_date
PK		FK → employees.employee_id		

Table: customers

customer_id	customer_name	city
PK		

Table: transactions

transaction_id	customer_id	transaction_date	transaction_type	amount
PK	FK → customers.customer_id			

Table: employee_history

employee_id	salary	effective_date

1. Department Salary Analysis

Which query correctly returns the employee count and average salary for each department?

- A) SELECT department_id, COUNT(*) AS employee_count, AVG(salary) AS average_salary FROM employees GROUP BY department_id;
- B) SELECT department_id, COUNT(*) AS employee_count, AVG(salary) AS average_salary FROM employees WHERE department_id;
- C) SELECT department_id, COUNT(*) AS employee_count, AVG(salary) AS average_salary FROM employees HAVING department_id;
- D) SELECT department_id, COUNT(*) AS employee_count, AVG(salary) AS average_salary FROM employees ORDER BY department_id;

2. High Average Salary Departments

Which query returns departments whose average salary is greater than 70000?

- A) SELECT department_id, AVG(salary) AS average_salary FROM employees GROUP BY department_id HAVING AVG(salary) > 70000;
- B) SELECT department_id, AVG(salary) AS average_salary FROM employees GROUP BY department_id WHERE AVG(salary) > 70000;
- C) SELECT department_id, AVG(salary) FROM employees WHERE AVG(salary) > 70000 GROUP BY department_id;
- D) SELECT department_id, AVG(salary) AS average_salary FROM employees HAVING AVG(salary) > 70000;

3. Employee Department Report

Which query correctly displays employee name, department name, and salary?

- A) SELECT e.employee_name, d.department_name, e.salary FROM employees e CROSS JOIN departments d;
- B) SELECT e.employee_name, d.department_name, e.salary FROM employees e LEFT JOIN departments d ON e.employee_id = d.department_id;
- C) SELECT e.employee_name, d.department_name, e.salary FROM employees e INNER JOIN departments d ON e.department_id = d.department_id;
- D) SELECT e.employee_name, d.department_name, e.salary FROM employees e INNER JOIN departments d ON e.employee_id = d.employee_id;

4. Employees Without Department

Which query finds employees whose department does not exist in the departments table?

- A) SELECT e.employee_name FROM employees e LEFT JOIN departments d ON e.department_id = d.department_id WHERE d.department_id IS NULL;
- B) SELECT e.employee_name FROM employees e LEFT JOIN departments d ON e.department_id =

d.department_id WHERE e.department_id IS NOT NULL;

C) SELECT e.employee_name FROM employees e INNER JOIN departments d ON e.department_id = d.department_id WHERE d.department_id IS NULL;

D) SELECT e.employee_name FROM employees e WHERE e.department_id IS NULL;

5. Employees With Multiple Projects

Which query finds employees assigned to more than 2 projects?

A) SELECT employee_id FROM projects GROUP BY employee_id WHERE COUNT(*) > 2;

B) SELECT employee_id FROM projects GROUP BY employee_id HAVING COUNT(*) > 2;

C) SELECT employee_id FROM projects WHERE COUNT(*) > 2 GROUP BY employee_id; D) SELECT employee_id FROM projects HAVING COUNT(*) > 2;

6. Total Project Cost

Which query calculates total project cost for each employee?

A) SELECT employee_id, project_cost FROM projects GROUP BY employee_id;

B) SELECT employee_id, SUM(project_cost) AS total_cost FROM projects GROUP BY employee_id;

C) SELECT employee_id, SUM(project_cost) AS total_cost FROM projects ORDER BY employee_id;

D) SELECT employee_id, SUM(project_cost) AS total_cost FROM projects WHERE project_cost > 0 GROUP BY employee_id;

7. Top 5 Salaries

Which query returns the 5 highest salaries?

A) SELECT * FROM employees LIMIT 5 ORDER BY salary DESC;

B) SELECT * FROM employees ORDER BY salary ASC LIMIT 5;

C) SELECT TOP 5 * FROM employees ORDER BY salary ASC;

D) SELECT * FROM employees ORDER BY salary DESC LIMIT 5;

8. Second-Highest Distinct Salary

Which query correctly returns the second-highest distinct salary?

A) SELECT MAX(salary) FROM employees WHERE salary > (SELECT MAX(salary) FROM employees);

B) SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);

C) SELECT MIN(salary) FROM employees WHERE salary < (SELECT MIN(salary) FROM employees); D) SELECT salary FROM employees ORDER BY salary DESC LIMIT 2;

9. Employees Above Overall Average

Which query finds employees whose salary is greater than the overall average salary?

- A) SELECT * FROM employees WHERE salary > (SELECT AVG(salary) FROM employees);
- B) SELECT * FROM employees WHERE AVG(salary) > salary;
- C) SELECT * FROM employees HAVING salary > (SELECT AVG(salary) FROM employees); D) SELECT * FROM employees WHERE salary > AVG(salary);

10. Departments With At Least 5 Employees

Which query returns departments having at least 5 employees?

- A) SELECT department_id FROM employees HAVING COUNT(*) >= 5;
- B) SELECT department_id FROM employees WHERE COUNT(*) >= 5 GROUP BY department_id;
- C) SELECT department_id FROM employees GROUP BY department_id HAVING COUNT(*) >= 5; D) SELECT department_id FROM employees GROUP BY department_id WHERE COUNT(*) >= 5;

11. Duplicate Employee Names

Which query identifies employee names appearing more than once?

- A) SELECT DISTINCT employee_name FROM employees GROUP BY employee_name;
- B) SELECT employee_name FROM employees WHERE COUNT(*) > 1 GROUP BY employee_name;
- C) SELECT employee_name FROM employees HAVING COUNT(*) > 1;
- D) SELECT employee_name FROM employees GROUP BY employee_name HAVING COUNT(*) > 1;

12. City-wise Employee Analysis

Which query returns cities having more than 10 employees?

- A) SELECT city FROM employees WHERE COUNT(*) > 10 GROUP BY city;
- B) SELECT city, COUNT(*) FROM employees WHERE city > 10 GROUP BY city;
- C) SELECT city FROM employees HAVING COUNT(*) > 10 GROUP BY city;
- D) SELECT city FROM employees GROUP BY city HAVING COUNT(*) > 10;

13. High Project Cost Employees

Which query finds employees whose total project cost exceeds 1,000,000?

- A) SELECT employee_id FROM projects HAVING SUM(project_cost) > 1000000;
- B) SELECT employee_id FROM projects WHERE SUM(project_cost) > 1000000 GROUP BY employee_id;
- C) SELECT employee_id, SUM(project_cost) FROM projects WHERE project_cost > 1000000 GROUP BY employee_id;
- D) SELECT employee_id FROM projects GROUP BY employee_id HAVING SUM(project_cost) > 1000000;

14. Employee Project Report

Which query correctly returns each employee's total project cost?

- A) SELECT e.employee_id, e.employee_name, p.project_cost FROM employees e JOIN projects p

ON e.employee_id = p.employee_id GROUP BY e.employee_id;

- B) SELECT e.employee_id, e.employee_name, SUM(p.project_cost) AS total_cost FROM employees e JOIN projects p ON e.employee_id = p.employee_id GROUP BY e.employee_id, e.employee_name;
- C) SELECT e.employee_id, e.employee_name, SUM(p.project_cost) AS total_cost FROM employees e JOIN projects p ON e.employee_id = p.project_id GROUP BY e.employee_id, e.employee_name;
- D) SELECT e.employee_id, e.employee_name, SUM(p.project_cost) FROM employees e CROSS JOIN projects p GROUP BY e.employee_id, e.employee_name;

15. High-Value Customers

Which query finds customers whose total transaction amount exceeds 200000?

- A) SELECT customer_id FROM transactions WHERE amount > 200000 GROUP BY customer_id; B) SELECT customer_id FROM transactions HAVING SUM(amount) > 200000 GROUP BY customer_id;
- C) SELECT customer_id FROM transactions GROUP BY customer_id HAVING SUM(amount) > 200000;
- D) SELECT customer_id, SUM(amount) FROM transactions GROUP BY customer_id WHERE SUM(amount) > 200000;

16. Monthly Transaction Analysis

Which query correctly calculates total transaction amount by year and month?

- A) SELECT YEAR(transaction_date), MONTH(transaction_date), SUM(amount) FROM transactions GROUP BY YEAR(transaction_date), MONTH(transaction_date);
- B) SELECT MONTH(transaction_date), SUM(amount) FROM transactions GROUP BY YEAR(transaction_date);
- C) SELECT YEAR(transaction_date), MONTH(transaction_date), SUM(amount) FROM transactions ORDER BY YEAR(transaction_date), MONTH(transaction_date);
- D) SELECT YEAR(transaction_date), MONTH(transaction_date), SUM(amount) FROM transactions GROUP BY transaction_date;

17. Credit Transaction Analysis

Which query calculates total CREDIT amount for each customer?

- A) SELECT customer_id, SUM(amount) FROM transactions GROUP BY customer_id HAVING transaction_type = 'CREDIT';
- B) SELECT customer_id, SUM(amount) FROM transactions GROUP BY transaction_type, customer_id;
- C) SELECT customer_id, amount FROM transactions WHERE transaction_type = 'CREDIT' GROUP BY customer_id;
- D) SELECT customer_id, SUM(amount) FROM transactions WHERE transaction_type = 'CREDIT' GROUP BY customer_id;

18. Customers With More Than 5 Transactions

Which query returns customers having more than 5 transactions?

- A) SELECT customer_id FROM transactions WHERE COUNT(*) > 5 GROUP BY customer_id;
- B) SELECT customer_id FROM transactions GROUP BY customer_id HAVING COUNT(*) > 5;
- C) SELECT customer_id FROM transactions HAVING COUNT(*) > 5;
- D) SELECT customer_id FROM transactions GROUP BY customer_id WHERE COUNT(*) > 5;

19. Customer Transaction Report

Which query correctly displays customer name and transaction amount?

- A) SELECT c.customer_name, t.amount FROM customers c JOIN transactions t ON c.customer_id = t.transaction_id;
- B) SELECT c.customer_name, t.amount FROM customers c JOIN transactions t ON c.customer_id = t.customer_id;
- C) SELECT c.customer_name, t.amount FROM customers c LEFT JOIN transactions t ON c.customer_name = t.customer_id;
- D) SELECT c.customer_name, t.amount FROM customers c CROSS JOIN transactions t;

20. Customers Without Transactions

Which query finds customers who have no transactions?

- A) SELECT c.customer_id FROM customers c LEFT JOIN transactions t ON c.customer_id = t.customer_id WHERE t.customer_id IS NULL;
- B) SELECT c.customer_id FROM customers c LEFT JOIN transactions t ON c.customer_id = t.customer_id WHERE c.customer_id IS NULL;
- C) SELECT c.customer_id FROM customers c INNER JOIN transactions t ON c.customer_id = t.customer_id WHERE t.customer_id IS NULL;
- D) SELECT c.customer_id FROM customers c RIGHT JOIN transactions t ON c.customer_id = t.customer_id WHERE t.customer_id IS NULL;

21. Highest Average Salary Department

Which query returns the department with the highest average salary?

- A) SELECT department_id, AVG(salary) AS avg_salary FROM employees GROUP BY department_id HAVING AVG(salary) = MAX(AVG(salary));
- B) SELECT department_id, MAX(AVG(salary)) FROM employees GROUP BY department_id;
- C) SELECT department_id, AVG(salary) AS avg_salary FROM employees GROUP BY department_id ORDER BY avg_salary DESC LIMIT 1;
- D) SELECT department_id, AVG(salary) AS avg_salary FROM employees ORDER BY avg_salary DESC LIMIT 1;

22. Minimum Salary by Department

Which query returns departments where the minimum salary is greater than 40000?

- A) SELECT department_id, MIN(salary) FROM employees HAVING MIN(salary) > 40000 GROUP BY department_id;
- B) SELECT department_id, MIN(salary) FROM employees GROUP BY department_id HAVING MIN(salary) > 40000;
- C) SELECT department_id, MIN(salary) FROM employees GROUP BY department_id WHERE MIN(salary) > 40000;
- D) SELECT department_id, MIN(salary) FROM employees WHERE MIN(salary) > 40000 GROUP BY department_id;

23. Employees Joined During 2026

Which query correctly finds employees who joined during calendar year 2026?

- A) SELECT * FROM employees WHERE joining_date >= '2026-01-01' AND joining_date < '2027-01-01';
- B) SELECT * FROM employees WHERE joining_date BETWEEN '2026-01-01' AND '2026-12-31';
- C) SELECT * FROM employees WHERE joining_date >= '2026-01-01' AND joining_date <= '2027-01-01';
- D) SELECT * FROM employees WHERE YEAR(joining_date) = 2026;

24. Most Recently Joined Employee

Which query returns the most recently joined employee?

- A) SELECT * FROM employees ORDER BY joining_date DESC LIMIT 1;
- B) SELECT * FROM employees ORDER BY joining_date ASC LIMIT 1;
- C) SELECT * FROM employees WHERE joining_date = MAX(joining_date); D) SELECT TOP 1 * FROM employees ORDER BY joining_date ASC;

25. Employees Working on Multiple Projects

Which query finds employees assigned to at least 3 distinct projects?

- A) SELECT employee_id FROM projects GROUP BY employee_id HAVING COUNT(DISTINCT project_id) >= 3;
- B) SELECT employee_id FROM projects WHERE COUNT(DISTINCT project_id) >= 3 GROUP BY employee_id;
- C) SELECT employee_id FROM projects HAVING COUNT(DISTINCT project_id) >= 3;
- D) SELECT employee_id FROM projects GROUP BY employee_id HAVING COUNT(project_id) > 3;

26. Employees With No Projects

Which query finds employees who are not assigned to any project?

- A) `SELECT e.employee_id FROM employees e INNER JOIN projects p ON e.employee_id = p.employee_id WHERE p.project_id IS NULL;`
- B) `SELECT e.employee_id FROM employees e LEFT JOIN projects p ON e.employee_id = p.employee_id WHERE p.project_id IS NULL;`
- C) `SELECT e.employee_id FROM employees e LEFT JOIN projects p ON e.employee_id = p.project_id WHERE p.project_id IS NULL;`
- D) `SELECT e.employee_id FROM employees e WHERE e.employee_id IS NULL;`

27. Highest-Cost Project

Which query returns the project with the highest project cost?

- A) `SELECT * FROM projects ORDER BY project_cost DESC LIMIT 1;`
- B) `SELECT * FROM projects ORDER BY project_cost ASC LIMIT 1;`
- C) `SELECT * FROM projects WHERE project_cost = MAX(project_cost);` D) `SELECT MAX(project_cost) FROM projects GROUP BY project_id;`

28. Departments Including Zero Employees

Which query includes departments even when they have zero employees?

- A) `SELECT d.department_id, d.department_name, COUNT(*) FROM departments d INNER JOIN employees e ON d.department_id = e.department_id GROUP BY d.department_id, d.department_name;`
- B) `SELECT d.department_id, d.department_name, COUNT(e.employee_id) AS employee_count FROM departments d LEFT JOIN employees e ON d.department_id = e.department_id GROUP BY d.department_id, d.department_name;`
- C) `SELECT d.department_id, d.department_name FROM departments d WHERE COUNT(e.employee_id) = 0;`
- D) `SELECT d.department_id, d.department_name, COUNT(e.employee_id) FROM departments d RIGHT JOIN employees e ON d.department_id = e.department_id GROUP BY d.department_id, d.department_name;`

29. Salary Range by Department

Which query calculates the salary range (maximum minus minimum) for each department?

- A) `SELECT department_id, MAX(salary) - MIN(salary) AS salary_range FROM employees;`
- B) `SELECT department_id, MAX(salary) + MIN(salary) AS salary_range FROM employees GROUP BY department_id;`
- C) `SELECT department_id, MAX(salary) - MIN(salary) AS salary_range FROM employees GROUP BY department_id;`

D) SELECT department_id, MIN(salary) - MAX(salary) AS salary_range FROM employees GROUP BY department_id;

30. Multiple Group Conditions

Which query returns departments having at least 5 employees and an average salary above 60000?

- A) SELECT department_id FROM employees GROUP BY department_id WHERE COUNT(*) >= 5 AND AVG(salary) > 60000;
- B) SELECT department_id FROM employees GROUP BY department_id HAVING COUNT(*) >= 5 OR AVG(salary) > 60000;
- C) SELECT department_id FROM employees WHERE COUNT(*) >= 5 AND AVG(salary) > 60000 GROUP BY department_id;
- D) SELECT department_id FROM employees GROUP BY department_id HAVING COUNT(*) >= 5 AND AVG(salary) > 60000;

31. Salary Ranking Within Department

Which window function correctly ranks employees by salary within each department?

- A) RANK() OVER (PARTITION BY department_id ORDER BY salary DESC)
- B) RANK() OVER (ORDER BY salary DESC)
- C) RANK(department_id) OVER (ORDER BY salary DESC)
- D) RANK() GROUP BY department_id ORDER BY salary DESC

32. Top 3 Salary Ranks Per Department

Which query returns employees whose salary is within the top 3 distinct salary ranks of their department?

- A) SELECT * FROM employees WHERE salary_rank <= 3;
- B) SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary DESC) AS salary_rank FROM employees e) x WHERE salary_rank < 3;
- C) SELECT * FROM (SELECT e.*, DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS salary_rank FROM employees e) x WHERE salary_rank <= 3;
- D) SELECT * FROM (SELECT e.*, RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS salary_rank FROM employees e) x WHERE salary_rank <= 3;

33. Running Salary Total

Which query calculates a running total of salary ordered by employee_id?

- A) SELECT employee_id, salary, SUM(salary) OVER (ORDER BY employee_id) AS running_total FROM employees;
- B) SELECT employee_id, salary, SUM(salary) OVER () AS running_total FROM employees;
- C) SELECT employee_id, salary, SUM(salary) GROUP BY employee_id AS running_total FROM employees;

D) SELECT employee_id, salary, SUM(salary) OVER (PARTITION BY employee_id) AS running_total FROM employees;

34. Department Average Alongside Employee Salary

Which query displays each employee's salary together with the average salary of that employee's department?

- A) SELECT employee_id, salary, AVG(salary) OVER (ORDER BY department_id) AS department_avg FROM employees;
- B) SELECT employee_id, salary, AVG(salary) GROUP BY department_id AS department_avg FROM employees;
- C) SELECT employee_id, salary, AVG(salary) OVER () AS department_avg FROM employees; D) SELECT employee_id, salary, AVG(salary) OVER (PARTITION BY department_id) AS department_avg FROM employees;

35. Salary Above Department Average

Which query finds employees earning more than their own department's average salary?

- A) SELECT * FROM employees WHERE salary > MAX(salary) OVER (PARTITION BY department_id);
- B) SELECT * FROM employees WHERE salary > (SELECT AVG(salary) FROM employees);
- C) SELECT * FROM employees e WHERE salary > AVG(salary) GROUP BY department_id;
- D) SELECT * FROM employees e WHERE salary > (SELECT AVG(salary) FROM employees WHERE department_id = e.department_id);

36. Latest Employee History Record

employee_history(employee_id, salary, effective_date) stores salary history. Which query returns the latest history record for each employee?

- A) SELECT * FROM employee_history ORDER BY effective_date DESC;
- B) SELECT * FROM (SELECT h.*, ROW_NUMBER() OVER (PARTITION BY employee_id ORDER BY effective_date DESC) AS rn FROM employee_history h) x WHERE rn = 1;
- C) SELECT * FROM employee_history WHERE effective_date = MAX(effective_date);
- D) SELECT employee_id, MAX(effective_date) FROM employee_history GROUP BY employee_id;

37. CTE-Based High Salary Analysis

Which query correctly uses a CTE to return employees earning above 80000?

- A) SELECT * FROM high_salary AS (SELECT * FROM employees WHERE salary > 80000);
- B) WITH employees AS (SELECT * FROM employees WHERE salary > 80000) SELECT * FROM employees;
- C) WITH high_salary AS (SELECT * FROM employees WHERE salary > 80000) SELECT * FROM high_salary;
- D) WITH high_salary SELECT * FROM employees WHERE salary > 80000;

38. Highest Average Salary Department

Which query returns the department having the highest average salary?

- A) SELECT department_id, AVG(salary) AS avg_salary FROM employees GROUP BY department_id ORDER BY avg_salary DESC LIMIT 1;
- B) SELECT department_id, AVG(salary) AS avg_salary FROM employees LIMIT 1;
- C) SELECT department_id, AVG(salary) AS avg_salary FROM employees GROUP BY department_id HAVING AVG(salary) = MAX(AVG(salary));
- D) SELECT department_id, MAX(AVG(salary)) FROM employees GROUP BY department_id;

39. Duplicate Employee Records

Which query identifies duplicate combinations of employee_id and joining_date?

- A) SELECT DISTINCT employee_id, joining_date FROM employees;
- B) SELECT employee_id, joining_date FROM employees WHERE COUNT(*) > 1 GROUP BY employee_id, joining_date;
- C) SELECT employee_id, joining_date FROM employees GROUP BY employee_id, joining_date HAVING COUNT(*) > 1;
- D) SELECT employee_id, joining_date FROM employees GROUP BY employee_id HAVING COUNT(*) > 1;

40. Unique Row Number for Duplicate Records

Which expression assigns a sequential row number within each employee_id based on joining_date?

- A) ROW_NUMBER() OVER (ORDER BY joining_date)
- B) ROW_NUMBER() OVER (PARTITION BY employee_id ORDER BY joining_date)
- C) ROW_NUMBER() GROUP BY employee_id ORDER BY joining_date
- D) ROW_NUMBER(employee_id) OVER (ORDER BY joining_date)

41. Highest Transaction Per Customer

Which query returns the highest transaction amount for each customer?

- A) SELECT customer_id, amount FROM transactions ORDER BY amount DESC LIMIT 1;
- B) SELECT customer_id, MIN(amount) AS highest_transaction FROM transactions GROUP BY customer_id;
- C) SELECT customer_id, MAX(amount) FROM transactions;
- D) SELECT customer_id, MAX(amount) AS highest_transaction FROM transactions GROUP BY customer_id;

42. Complete Highest Transaction Record

Which query returns the complete highest-value transaction record for every customer?

- A) SELECT customer_id, MAX(amount) FROM transactions GROUP BY customer_id;
- B) SELECT * FROM transactions WHERE amount = MAX(amount);
- C) SELECT * FROM transactions ORDER BY amount DESC;
- D) SELECT * FROM (SELECT t.*, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY amount DESC) AS rn FROM transactions t) x WHERE rn = 1;

43. Monthly Transaction Revenue

Which query calculates total transaction amount for each year and month?

- A) SELECT YEAR(transaction_date), MONTH(transaction_date), SUM(amount) FROM transactions GROUP BY transaction_date;
- B) SELECT YEAR(transaction_date), SUM(amount) FROM transactions GROUP BY MONTH(transaction_date);
- C) SELECT YEAR(transaction_date) AS yr, MONTH(transaction_date) AS mon, SUM(amount) AS total_amount FROM transactions GROUP BY YEAR(transaction_date), MONTH(transaction_date);
- D) SELECT MONTH(transaction_date), SUM(amount) FROM transactions GROUP BY transaction_date;

44. Monthly Customer Ranking

Which window expression ranks customers by their monthly transaction total within each year/month?

- A) RANK() OVER (ORDER BY SUM(amount) DESC)
- B) RANK() OVER (PARTITION BY YEAR(transaction_date), MONTH(transaction_date) ORDER BY SUM(amount) DESC)
- C) RANK() OVER (GROUP BY YEAR(transaction_date), MONTH(transaction_date) ORDER BY SUM(amount) DESC)
- D) RANK() OVER (PARTITION BY customer_id ORDER BY SUM(amount) DESC)

45. Running Transaction Total Per Customer

Which query calculates a running transaction total separately for each customer?

- A) SELECT customer_id, transaction_date, amount, SUM(amount) OVER (PARTITION BY customer_id ORDER BY transaction_date) AS running_total FROM transactions;
- B) SELECT customer_id, transaction_date, amount, SUM(amount) OVER (PARTITION BY transaction_date ORDER BY customer_id) AS running_total FROM transactions;
- C) SELECT customer_id, transaction_date, amount, SUM(amount) GROUP BY customer_id AS running_total FROM transactions;
- D) SELECT customer_id, transaction_date, amount, SUM(amount) OVER (ORDER BY transaction_date) AS running_total FROM transactions;

46. Cumulative Project Cost

Which query calculates cumulative project cost ordered by project start date?

- A) `SELECT project_id, start_date, project_cost, SUM(project_cost) OVER () AS cumulative_cost FROM projects;`
- B) `SELECT project_id, start_date, project_cost, SUM(project_cost) OVER (ORDER BY start_date) AS cumulative_cost FROM projects;`
- C) `SELECT project_id, start_date, project_cost, SUM(project_cost) GROUP BY start_date AS cumulative_cost FROM projects;`
- D) `SELECT project_id, start_date, project_cost, SUM(project_cost) OVER (PARTITION BY project_id ORDER BY start_date) AS cumulative_cost FROM projects;`

47. Previous Transaction Amount

Which query gets the previous transaction amount for each customer?

- A) `SELECT customer_id, transaction_date, amount, LAG(amount) OVER (ORDER BY transaction_date) AS previous_amount FROM transactions;`
- B) `SELECT customer_id, transaction_date, amount, LAG(amount) GROUP BY customer_id AS previous_amount FROM transactions;`
- C) `SELECT customer_id, transaction_date, amount, LEAD(amount) OVER (PARTITION BY customer_id ORDER BY transaction_date) AS previous_amount FROM transactions;`
- D) `SELECT customer_id, transaction_date, amount, LAG(amount) OVER (PARTITION BY customer_id ORDER BY transaction_date) AS previous_amount FROM transactions;`

48. Next Transaction Amount

Which query gets the next transaction amount for each customer?

- A) `SELECT customer_id, transaction_date, amount, LAG(amount) OVER (PARTITION BY customer_id ORDER BY transaction_date) AS next_amount FROM transactions;`
- B) `SELECT customer_id, transaction_date, amount, LEAD(amount) OVER (ORDER BY transaction_date) AS next_amount FROM transactions;`
- C) `SELECT customer_id, transaction_date, amount, LEAD(amount) OVER (PARTITION BY customer_id ORDER BY transaction_date) AS next_amount FROM transactions;`
- D) `SELECT customer_id, transaction_date, amount, LEAD(amount) GROUP BY customer_id AS next_amount FROM transactions;`

49. Transaction Data Quality Check

Which condition identifies transactions with a missing customer, missing amount, or negative amount?

- A) `WHERE amount IS NOT NULL OR customer_id IS NOT NULL OR amount >= 0`
- B) `WHERE amount IS NULL AND customer_id IS NULL AND amount < 0`
- C) `WHERE amount IS NULL OR customer_id IS NULL OR amount < 0`

D) WHERE amount = NULL OR customer_id = NULL OR amount < 0

50. Top 2 Salary Ranks Per Department Including Ties

Which query returns employees in the top 2 salary ranks of each department, including ties?

- A) SELECT department_id, MAX(salary) FROM employees GROUP BY department_id;
- B) SELECT * FROM (SELECT e.*, DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS rnk FROM employees e) x WHERE rnk <= 2;
- C) SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary DESC) AS rnk FROM employees e) x WHERE rnk <= 2;
- D) SELECT * FROM employees WHERE salary IN (SELECT MAX(salary) FROM employees);